# CS4094 MS3 Progress Report: Distributed DAG Task Scheduler

MS2 could complete jobs as long as workers stayed alive. If a worker crashed mid-task, however, the scheduler kept that assignment indefinitely. MS3 adds leases and worker heartbeats so unfinished work can return to the queue. We also added fault-injection tests, measured recovery under crashes, and fixed memory growth caused by idle worker polling.

The specification defines the properties discussed here. The test matrix lists the test cases for each property. The run logs and exported histories are in results/ms3-logs/.

## A. Verification results

The full test run we submitted on October 29 is `results/ms3-logs/make-test-3.log`, from clean source `966965e`. Maven verification passed 77 JUnit tests, and pytest passed 102 tests with no failures or expected failures (exit code 0). The pytest XML is `results/ms3-logs/junit-make-test-3.xml`. This establishes the result for that revision, not for every change made afterward.

A later independent local run passed 101 pytest tests, but the chaos test failed. Its crash and pause were timed with fixed sleeps, and in that run both landed while the victims held no task, so no lease could expire; the failed history has no `lease_expired` event. The test now holds the crash victim in a running attempt with a test gate, freezes the second worker only while it holds a task, and requires those attempts to expire. The failed run and the reruns are in `results/ms3-logs/independent-check-ahmad/`.

The final full run, `results/ms3-logs/make-test-5.log` from clean source `85b0ecb`, includes the chaos-test correction and the Hokea fix. Maven verification passed 77 JUnit tests; pytest ran 107 tests, of which 105 passed and 2 were skipped, with no failures or expected failures (exit code 0). The skipped tests check the Hokea adapter against a real Hokea installation, which the test container does not have; they pass in the course lab environment. The run's service-backed tests exported 53 evidence directories under `results/ms3-logs/evidence-make-test-5/`, and the offline history checker found no safety violations in those 53 job histories. This checks the recorded executions; it does not cover every possible execution.

The main claims map to evidence as follows:

- S1 (accepted success only): `LeaseTest` and `tests/faults/test_network_faults.py` exercise expired attempts and late reports. The saved histories are also checked for success after expiry.
- S2 (dependency ordering): the integration tests in `tests/integration/` cover chains, joins, and concurrent branches. `check_history` verifies recorded assignment and start ordering.
- S3 (single accepted completion): duplicate-report, lost-reply, restart, and pause tests cover retries and old attempts. `LeaseTest` includes an eight-thread race. The history checker also rejects duplicate success.
- L1 (progress with available workers): functional workload, slow-task, and short-partition tests check that ready work completes and long-running tasks are not expired while renewals continue.

- L2 (worker-failure recovery): `tests/faults/test_worker_crash.py::test_worker_crash_reassignment` is the MS2 crash oracle, now passing. Additional cases in `tests/faults/test_lease_faults.py` cover crashes before start, during an operation, and after upload; restart and pause cases cover other losses of ownership. These also check the P-REC bound when an idle worker is polling. `tests/faults/test_crash_rerun_bounded.py` is the test requested in the MS2 feedback: a worker is killed during an ordinary 20-second task, with no test hooks, and the task must run again within 60 seconds and succeed once. It and the crash oracle also passed on the course cluster (Section F).

The full mapping, including tests of the checker itself, is in test-matrix.md. Each fault test saves its scheduler history, worker events, and applicable fault or proxy logs. The event and verification formats are described in evidence.md.

Earlier full runs remain in the archive. `make-test-1` had one failure: a counters test still expected MS2's exact metric list. `make-test-2` passed all 102 pytest tests but ran before the idle-memory fix. Its `+uncommitted` label came from writing the log inside the repository. `make-test-4` ran the Hokea fix and an earlier version of the chaos correction before the documentation review was merged (105 passed, 2 skipped). Run 5 is the full-suite result used for the property summary above.

## B. What changed

The scheduler grants each attempt a lease, five seconds by default. Start acknowledgments and heartbeats renew it. When the lease expires, the scheduler releases the owner, records `lease_expired`, and returns the task to the end of the ready queue. Expiry is checked on claims, starts, heartbeats, and reports, as well as during a scan every 100 ms. Replayed starts and reports from expired attempts receive `409 LEASE_EXPIRED`.

Workers now send heartbeats while holding an attempt. The interval comes from `/v1/health` and defaults to one-fifth of the lease. If a renewal is rejected with `409`, the worker cancels the operation and abandons the attempt without a new upload or result report. Test-only gates can stop a worker before start, during execution, or before reporting.

We added `tests/harness/netfault.py` to inject dropped, delayed, and repeated messages or partition a worker's link to the scheduler. The harness also supports worker crashes and pauses. The history checker now detects success from an expired attempt and expiry recorded for a non-owner.

The new coverage includes 16 lease JUnit tests using an injected clock, one idle-polling JUnit test, 14 fault-injection tests (one is the crash test requested in the MS2 feedback), and two checker-falsification tests. Together with the original crash oracle, the suite contains 15 fault tests. Recovery benchmarks are implemented in `benchmarks/recovery.py` and run through `make bench-recovery`.

Two bugs were found during testing. A replayed start could return a cached acknowledgment after expiry. The report path already rejected its result, but the start response was inconsistent with the lease rule. The restart test exposed this, and the JUnit suite now covers it.

The second bug was idle-memory growth, inherited from MS2. Four idle workers caused the live heap to grow by about 257 MiB per hour because each empty claim created a new history event and receipt. Empty claims no longer append history events; each session replaces its previous idle receipt.

Polling counts are still available at `/v1/metrics`. The follow-up probe showed no continuing idle growth. Logs are in `results/ms3-logs/memory-probe/`.

After the first submission we fixed two more problems. The Hokea adapter refused to run on the course cluster (Section F), and the chaos test could fail without any system fault (Section A).

## C. Design decisions

We use leases because the scheduler cannot reliably tell whether a worker has crashed or is just slow. Instead, it asks whether that worker has renewed its assignment in time. Using the scheduler's monotonic clock means we do not have to compare clocks on different machines.

A new assignment gets a higher attempt number. The scheduler accepts results only from the current attempt, and each attempt writes to its own artifact keys. This matters when an old worker resumes after the task has already been reassigned: its work may finish, but it cannot replace the accepted output.

We chose a five-second lease and a one-second heartbeat interval. A shorter lease would reclaim tasks faster, but it could also expire healthy workers when messages are slow. Three lost heartbeats fit within the default lease when requests are prompt; delays or timeouts reduce that margin. We also check expiry when a report arrives, not just on the background scan, so a late success is not accepted simply because the scan has not run yet.

We left deadline decisions to the scheduler rather than trying to estimate them on each worker. A disconnected worker can keep running until it receives a rejection. Attempt checks protect the scheduler's accepted result, but they do not protect external side effects such as paid API requests. The scheduler also remains a single in-memory process, so its crash still loses the run.

## D. Fault-test observations

The final test run recorded 11 lease expiries across the fault tests (run 3 recorded ten). Detection took 5.01-5.11 seconds after the last accepted renewal, never before the five-second lease. With an idle worker polling, reassignment took 5.01-5.11 seconds. The chaos test took 5.87-6.32 seconds because work was already queued. Fault-free, slow-task, and short-partition tests recorded no expiry. Tests rejected or abandoned late work and retained one accepted success per task.

We checked the tests against deliberately introduced defects. Disabling renewal caused two lease tests to fail; skipping the expiry check when processing reports caused two others to fail. The checker-falsification tests modify recorded histories to add invalid events and confirm that the checker rejects them. These checks are described in the test matrix.

## E. Recovery benchmarks

The benchmark uses six-task jobs with 100 ms fixture operations, four warmup jobs, and 24 measured jobs per run. W is fixed at four workers; C is the maximum number of jobs in flight (1, 4, or 16). Each configuration was run three times on fresh Compose deployments, with one CPU and 512 MiB per service. Baseline mode injects no failures. Crash mode requests two kills of busy workers, after the sixth and fourteenth measured job completions, and starts replacements.

The selected benchmark source is `d4fcda9`; its service code is the same as tested source `966965e`. The detailed per-run values and recovered attempts are in `results/ms3-logs/recovery-benchmark/`, especially `runs.csv`, `recoveries.csv`, `recovery_jobs.csv`, and `summary.csv`. Percentiles below use the nearest-rank method over complete runs only.

Increasing C from 1 to 4 raised fault-free throughput from 4.40 to 14.78 tasks/s with the same four workers. That is consistent with having enough independent jobs to keep workers busy, although we did not measure worker utilization directly. At C = 16, mean throughput was 11.85 tasks/s rather than continuing to rise, and the standard deviation was 6.12 tasks/s. We cannot tell from these runs whether the scheduler, workers, or shared host limited throughput. What we can say is that higher concurrency did not produce a measured throughput improvement beyond C = 4.

Job completion time also increased at C = 16. In the fault-free runs, median job time rose from 1,563 ms at C = 4 to 5,851 ms at C = 16, and p95 rose from 1,805 ms to 20,143 ms. More jobs were competing for the same four workers, but these measurements do not isolate queueing from other resource contention. Crashes added a different delay: affected tasks had to wait for their leases to expire before another worker could take over. The detailed results follow.

At C = 1, all three baseline runs completed, averaging 4.40 +/- 2.23 tasks/s. Their job completion times were 1,042 ms at p50 and 5,207 ms at p95. Two of three crash runs completed, averaging 4.03 +/- 0.01 tasks/s. Their job times were 1,035 ms at p50 and 6,250 ms at p95. Four recoveries had a 5,004 ms median and 5,045 ms maximum.

At C = 4, all three baseline and three crash runs completed. Throughput averaged 14.78 +/- 1.33 tasks/s without faults and 9.66 +/- 0.24 tasks/s with crashes. Baseline job times were 1,563/1,805 ms (p50/p95), compared with 1,690/6,656 ms in crash mode. Five recoveries had a 5,058 ms median and 5,103 ms maximum.

At C = 16, all six runs completed. Baseline throughput was 11.85 +/- 6.12 tasks/s; crash throughput was 9.92 +/- 5.16 tasks/s. Baseline job times were 5,851/20,143 ms (p50/p95), compared with 6,006/28,598 ms under crashes. Four recoveries had a 5,052 ms median. One task waited in the ready queue while all workers were busy, making the maximum recovery time 10,328 ms.

At C = 4, mean throughput fell from 14.78 to 9.66 tasks/s under crashes, about a 35% reduction in these runs. At C = 1 and C = 16, the throughput differences are harder to interpret because of run-to-run variation. We did not measure a specific CPU, lock, or network bottleneck, so we cannot assign the slowdown to one component.

Across the eight completed crash runs, the driver requested 16 kills of executing workers. Three targets had already reported by the time the kill completed; the other 13 left unfinished tasks. All 13 recovered. Detection took 5.00-5.09 seconds. Twelve tasks were reassigned within 64 ms of expiry, for total recovery of 5.00-5.10 seconds. The remaining task waited roughly 5.3 seconds in the FIFO queue. So the observed recoveries meet the stated P-REC condition when an idle worker is available; the 10.3-second case does not violate that bound because every worker was occupied.

Jobs directly affected by kills had median completion times of 6.3, 6.9, and 11.4 seconds at C = 1, 4, and 16. Other jobs in those runs had medians of 1.0, 1.7, and 6.0 seconds. Unaffected-job medians differed by less than 8% from the baseline: 1.03 versus 1.04 seconds, 1.68 versus 1.56, and 5.98 versus 5.85. This suggests that the main cost to affected jobs was waiting for expiry and repeating work; it does not rule out indirect effects on other jobs.

The host was noisy enough to limit comparisons with MS2. For W = 4, fault-free throughput was 4.4, 14.8, and 11.9 tasks/s at C = 1, 4, and 16; Step 12 recorded 6.0, 15.4, and 16.7. At C = 1, one baseline run achieved 1.8 tasks/s and the next two achieved 5.7. A same-day unchanged-MS2-code control reached 3.06 tasks/s, close to the MS3 control range of 2.99-3.30 tasks/s. See `results/ms3-logs/perf-attribution/`. This supports host variability as an explanation without excluding code-related changes at other loads.

Only 17 of 18 planned benchmark runs completed. Crash C = 1, repetition 1 ended when the driver's six-second HTTP event-stream read timed out. Its history shows that all 27 submitted jobs completed and both kills recovered, but the driver did not finish its planned measured phase. We excluded that run from the statistics. The offline checker accepted available histories from all 18 runs, covering 503 jobs. The benchmark archive omits large worker and Compose logs, so that check uses saved scheduler histories and snapshots.

The archive also retains three aborted benchmark attempts: one targeted idle workers, one stopped after the memory fix changed the code, and one exposed a replacement-worker naming race in the driver. Their status and source revisions are documented in `results/ms3-logs/README.md`.

Measurements were taken on the MS2 laptop: Windows 11, 12 CPUs, 16 GB RAM, Docker Desktop 27.1.1, and a Linux VM capped at 3 GB. The browser and IDE shared the host. Fixture tasks often finish before the first heartbeat, so this workload adds little heartbeat traffic. The results describe a synthetic workload on one host, not a minimum throughput guarantee. The memory probe suggests roughly 10 KB retained per completed task, or about 11,000 tasks with a 128 MiB heap; that is an estimate, not a hard capacity limit.

## F. Deployment

`make up` starts the scheduler, artifact store, and three workers in Compose, with no automatic restart and the resource caps noted above. `make demo` runs the arithmetic and video workloads.

The Hokea adapter includes the fault proxy and requires zero expected failures. In the first course-cluster attempt, Kubernetes accepted the runner Job and pytest collected three tests, but all three stopped during setup with `Hokea source mismatch at check.py` (`results/handoffs/step-11/acceptance-hokea-20261003T200759-f99f027a/CLUSTER-RESULT.md`).

The cause was our adapter, as the MS2 feedback said. It required the installed Hokea package to match public commit `427b946` byte for byte, and the course runner image ships its own build, which differs from that commit in 7 of its 14 files. We reproduced the error on our namespace on October 30 with the unchanged code. The adapter now records the installed files' hashes in each test's evidence and stops only if Hokea lacks a class, method or option the adapter calls.

The same day, `hokea test` passed all four cluster tests in 2 minutes 11 seconds (source `611c1de`, with images built from the unchanged service code at `6104fa7`): the smoke test, the functional workload, the crash oracle and the bounded re-run test, with worker crashes done by force-deleting the pod. The crash oracle's task was reassigned 5.57 seconds after its last heartbeat; in the bounded test the task ran again 4.5 seconds after its pod was deleted and succeeded once. All seven job histories passed the offline checker, and no pods were left running. The files that run on the cluster are unchanged in the final source. The logs are in `results/ms3-logs/hokea-cluster/`. The other fault tests need the in-process proxy or SIGSTOP, so they still run only locally.

## G. Known limitations

Scheduler state is not durable. A crash loses queued and in-flight work, and restarting creates a new run. Retaining full histories in memory also limits run length. With the default 128 MiB heap, the memory probe suggests around 11,000 completed tasks, or roughly 10-15 minutes at sustained full load.

A live worker may lose its lease during a long pause or partition, causing work to repeat. Only one completion can be accepted, but external side effects need their own idempotency. There is no retry limit, so a permanently failing operation leaves its job RUNNING. The scheduler trusts workers rather than recomputing their output. Network fault injection covers worker-to-scheduler links, not the artifact store; the store is assumed available; it keeps its object index in memory, and we do not rely on its files surviving a restart (on Hokea they do not).

## H. Later milestones

Possible MS4-MS5 work includes responding to red-team findings, adding a retry limit and terminal FAILED state, storing scheduler state durably, and running the remaining fault suites under Compose and Hokea. None of these is an MS3 guarantee.